

TransSGPN: Transistor Network Synthesis via Switching Graph Policy Network

Anonymous submission

Abstract

Transistor network synthesis—finding a compact CMOS switching network that correctly implements a given Boolean function—is a central step in custom-cell design. Existing exact and SAT-based methods are powerful for small functions but scale poorly as the number of variables grows and cannot generalize across functions without re-solving from scratch. We present TransSGPN, a learning-based framework that casts transistor network synthesis as a sequential graph-generation Markov decision process (MDP) and trains a Switching Graph Policy Network (SGPN) to solve it. The policy operates over a unified graph comprising both the partial synthesized network and a fixed SOP-based compensate network that encodes the target Boolean function, allowing the GNN backbone to reason jointly about coverage obligations and existing topology at every step. Training proceeds in three phases: Phase 1 performs supervised NLL pretraining on a dataset of known-optimal transistor networks to initialize the policy; Phase 2 applies PPO-based reinforcement learning with a GRPO group baseline and an adaptive self-tightening reward to push the policy toward minimum transistor count; Phase 3 extends RL to joint pull-down/pull-up network (PDN/PUN) synthesis with a physical placement reward derived from ASTRAN, a standard-cell layout tool, to optimize cell width (CPP) directly. On a benchmark of [TODO: N] Boolean functions with up to three variables, TransSGPN achieves [TODO: X%] synthesis success rate and matches or improves upon the reference transistor count on [TODO: Y%] of functions; physical-aware finetuning further reduces the ASTRAN placement objective by [TODO: Z%] compared to the Phase 2 baseline, demonstrating that a single policy can jointly optimize logical correctness, transistor count, and physical layout quality.

Introduction

Transistor network synthesis is a foundational step in CMOS standard-cell design. Given a Boolean function f , the goal is to build a minimum-transistor switching network—an undirected graph whose edges represent transistors and whose source-to-sink connectivity implements f . The synthesized topology directly determines transistor count, diffusion-sharing opportunities, and downstream layout quality; even

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

a single extra transistor can degrade area, power, and timing after placement and routing.

Limitations of existing methods. Exact SAT-based methods [TODO: Authors](TODO: Year)b), [TODO: Authors](TODO: Year)a) can provably find minimum-transistor solutions but solve each function from scratch, with no reuse across instances. Runtime grows exponentially with the number of variables, and the methods offer no generalization to unseen functions. MCTS-based approaches [TODO: Authors](2026) improve scalability but are still per-instance search procedures that cannot internalize cross-function structural patterns. Neither family of methods *learns* a generalizable synthesis policy, so every new function incurs the full cost of a fresh optimization.

Our approach: learned sequential graph generation. We treat transistor network synthesis as a sequential graph-generation Markov decision process (MDP) and train a policy to solve it. At each step the policy places one transistor, guided by a graph neural network that observes the current partial circuit together with a compact representation of the target function. A single policy trained over a distribution of Boolean functions generalizes at inference time to new functions without any per-instance search.

Key technical ideas. (1) **Theoretical guarantees by construction.**

We prove a *Monotonicity* theorem showing that adding transistors never removes existing conducting paths, and derive a *Safety Inheritance* corollary that reduces safety verification at each step from checking the whole network to a single BFS on the new edge. Safety and correctness are therefore maintained as invariants throughout the entire trajectory, not checked post-hoc.

(2) **A novel GNN architecture for transistor synthesis.** We design TransSGPN, whose core is the Switching Graph Policy Network (SGPN) — a five-head factored policy built on a Message Passing Neural Network (MPNN) [Gilmer et al.(2017)Gilmer, Schütt, Mayr et al.] backbone. Each head specializes in one sub-decision of the action (source node, drain node, control variable, polarity), with safety masks applied independently at each head. A learnable virtual-node embedding allows the policy to propose fresh topology without pre-committing to a node count. All heads share a single MPNN forward pass over a

unified graph that merges the partial synthesized network with a fixed SOP compensate network.

(3) Three-phase training toward minimum transistor count and physical quality. Phase 1 NLL pretraining on known-optimal solutions initializes the policy. Phase 2 PPO finetuning with a GRPO group-relative baseline, an adaptive self-tightening reward, and an entropy bonus drives convergence toward minimum transistor count without mode collapse. Phase 3 extends the RL loop to joint PDN+PUN synthesis with a physical placement reward from AS-TRAN [Ziesemer and Reis(2014)], directly optimizing the weighted placement objective (cell width, gate mismatch, wire length, routing density, and diffusion gaps).

Contributions.

- Theoretical foundations for sequential transistor synthesis.** We prove a Monotonicity theorem (adding transistors is order-invariant for conducting paths) and derive a Safety Inheritance corollary that makes safety checking $O(|P^-| \cdot |V|)$ per transistor candidate — independent of the number of already-placed transistors. Together they guarantee that any trajectory produced under our action masks is *correct and safe by construction*, with no post-hoc verification needed (Section).
- A new five-head architecture for transistor network synthesis.** TransSGPN decomposes transistor placement into four factored sub-decisions, each handled by a specialized MLP head sharing an MPNN backbone. Key design choices include: (a) a *unified graph* state that fuses the partial circuit G_t with a fixed SOP compensate network C encoding the target function, giving the GNN simultaneous access to current topology and remaining coverage obligations; (b) a *virtual new-node embedding* that extends the node space dynamically, allowing the policy to create fresh internal nodes without an explicit node-count decision; (c) *per-head safety masking* applied independently to the node, variable, and polarity heads, so that the generated action is guaranteed safe without additional filtering; (d) a *batched PackedGraph forward pass* that packs all trajectory steps into one MPNN call, replacing $O(K \cdot T_{\max})$ sequential forward passes with one (Section).
- Three-phase training: logical and physical optimization.** Phase 1 NLL pretraining initializes the policy on known-optimal solutions. Phase 2 PPO finetuning with a GRPO group-relative baseline, adaptive self-tightening reward $r = T^*(f)/T$, and entropy bonus converges toward minimum transistor count. Phase 3 extends RL to *joint PDN+PUN synthesis* with a physical placement reward, bridging logical synthesis and layout quality in a single learned policy (Section).
- Physical-aware joint synthesis via ASTRAN reward.** For a target function f , we simultaneously synthesize the pull-down network (PDN, implementing $\neg f(\mathbf{x})$) and pull-up network (PUN, implementing $f(\neg \mathbf{x})$) using the same policy. Each complete PDN+PUN pair is evaluated by ASTRAN [Ziesemer and Reis(2014)] — a standard-cell layout tool — which returns five placement metrics:

cell width (CPP), gate mismatches, wire length, routing density, and diffusion cuts. Their weighted sum serves as the joint RL reward, aligning the policy’s objective with ASTRAN’s own SA optimization (Section).

- Empirical validation.** We evaluate on a benchmark of [TODO: N] Boolean functions with up to three variables. Phase 2 achieves [TODO: X%] synthesis success rate and matches the minimum transistor count on [TODO: Y%] of functions. Phase 3 further reduces the ASTRAN placement objective by [TODO: Z%] compared to the Phase 2 baseline, at [TODO: W $\times$] the speed of SAT-based exact methods (Section).

The remainder of the paper is organized as follows. Section introduces transistor network synthesis and placement/routing background. Section presents our theoretical foundations and the TransSGPN architecture with three-phase training. Section reports experimental results and ablations. Section concludes.

Preliminaries

Transistor Networks and Switching Graphs

Let $\mathbf{x} = (x_1, \dots, x_N)$ be N Boolean input variables and $\pi \in \{0, 1\}^N$ an input pattern. The *literal alphabet* is $\mathcal{L} = \{x_1, \neg x_1, \dots, x_N, \neg x_N\}$.

Definition 1 (Switching Graph). A *switching graph* is an undirected multigraph $H = (V, E)$ with two distinguished terminals $\text{SRC}, \text{SNK} \in V$. Each edge $e \in E$ is a *switch* that is either open or closed; the graph *conducts* $\text{SRC} \rightarrow \text{SNK}$ iff a path of closed switches connects SRC to SNK.

Definition 2 (Transistor Network). A *transistor network* is a switching graph $G = (V, E)$ in which each switch $(u, v, \ell) \in E$ is a transistor controlled by a literal $\ell \in \mathcal{L}$: the transistor is closed (conducts) under pattern π iff $\ell(\pi) = 1$.

Let $P^+ = f^{-1}(1)$ and $P^- = f^{-1}(0)$ for Boolean function $f : \{0, 1\}^N \rightarrow \{0, 1\}$. A transistor network G *correctly implements* f iff:

$$\begin{aligned} \forall \pi \in P^+ : G \text{ conducts } \text{SRC} \rightarrow \text{SNK} \text{ under } \pi, & \text{ (Coverage)} \\ \forall \pi \in P^- : G \text{ does not conduct } \text{SRC} \rightarrow \text{SNK} \text{ under } \pi. & \text{ (Safety)} \end{aligned}$$

The *synthesis problem* is to find a correct implementation with minimum transistor count: $\min\{|E| : G \text{ correctly implements } f\}$.

Prior work. Exact SAT-based solvers [TODO: Authors](TODO: Year)b), [TODO: Authors](TODO: Year)a) optimally solve each function independently but scale exponentially with N and provide no cross-function generalization. MCTS-based approaches [TODO: Authors](2026)] improve scalability but remain per-instance search procedures. No existing method learns a generalizable synthesis policy.

CMOS Cell Synthesis as Dual Switching Network Problems

A CMOS standard cell implementing $f(\mathbf{x})$ consists of two complementary switching networks realized with different transistor types:

Definition 3 (PDN and PUN). The *pull-down network* (PDN) is an NMOS transistor network connected between GND and the output ZN. The *pull-up network* (PUN) is a PMOS transistor network connected between VDD and ZN. The cell is correct iff the PDN pulls ZN low when $f = 0$ and the PUN pulls ZN high when $f = 1$.

The following proposition shows that synthesizing a CMOS cell for f decomposes exactly into two independent switching network synthesis problems.

Proposition 1 (Dual Switching Network Equivalence). *A CMOS cell correctly implements $f(\mathbf{x})$ if and only if:*

1. *The PDN correctly implements $\neg f(\mathbf{x})$ as a switching network, i.e., PDN on-patterns = P^- .*
2. *The PUN correctly implements $f(\neg \mathbf{x})$ as a switching network, i.e., PUN on-patterns = $\{\neg \pi : \pi \in P^+\}$.*

Proof. (PDN) An NMOS transistor with gate x_i conducts when $x_i = 1$, so NMOS network semantics are identical to the switching network model. The PDN must pull ZN low when $f(\mathbf{x}) = 0$, i.e., it must conduct for all $\pi \in P^-$. It must not pull low when $f(\mathbf{x}) = 1$, i.e., it must not conduct for $\pi \in P^+$. Hence the PDN implements $\neg f(\mathbf{x})$ with on-patterns = P^- .

(PUN) A PMOS transistor with gate x_i conducts when $x_i = 0$, which is equivalent to a switch controlled by $\neg x_i$. Substituting $y_i = \neg x_i$, the PMOS network conducts under $\mathbf{y}$ iff $f(\neg \mathbf{y}) = 1$ — that is, iff $\mathbf{y} \in \{\neg \pi : \pi \in P^+\}$. In terms of the original signal $\mathbf{x} = \neg \mathbf{y}$, the PUN must conduct when $f(\mathbf{x}) = 1$ with complemented control signals, implementing $f(\neg \mathbf{x})$ with on-patterns = $\{\neg \pi : \pi \in P^+\}$. $\square$

Proposition 1 has a key consequence for our framework: *synthesizing a full CMOS cell is equivalent to independently solving two standard switching network synthesis problems* — one for PDN and one for PUN. Both problems share the same Boolean structure and can be solved by the same policy; only their on/off pattern sets differ. This is the foundation for the joint PDN+PUN training in Phase 3 of TransSGPN.

Placement and routing. After synthesis, transistors are placed into a standard-cell layout: NMOS (PDN) in the bottom row, PMOS (PUN) in the top row, each sharing a common output node ZN and separated by routing tracks. Key physical metrics include cell width in *contacted poly pitch* (CPP) units, gate mismatches between the P and N rows, wire length, routing density, and diffusion cuts (gaps in the Euler path). We use ASTRAN [Ziesemer and Reis(2014)] to evaluate placement quality via a weighted objective Φ (defined in Section), and minimize Φ directly as the RL reward in Phase 3.

The TransSGPN Framework with Switching Graph Policy Network

Theoretical Foundations

We establish four results that jointly guarantee both the *soundness* (every generated circuit is valid) and *completeness* (every valid circuit is reachable) of our safety-masked sequential generation framework.

Theorem 2 (Monotonicity of Conducting Paths). *For any transistor network G and any transistor $e \notin E(G)$, $\text{Paths}(G) \subseteq \text{Paths}(G \cup \{e\})$.*

Proof. Adding e can only activate new conducting channels; it cannot deactivate any already-conducting transistor. Hence every path that conducted in G still conducts in $G \cup \{e\}$. $\square$

Corollary 3 (Incremental Safety Checking). *If G is safe, then $G \cup \{(u, v, \ell)\}$ is safe iff (u, v, ℓ) alone does not create a new $\text{SRC} \rightarrow \text{SNK}$ path under any $\pi \in P^-$.*

Proof. Follows directly from Theorem 2: any P^- -path created in $G \cup \{e\}$ must use e (since G was already safe), so checking e alone suffices. $\square$

This reduces safety verification per candidate from $O(|E|)$ to $O(|P^-| \cdot |V|)$, independent of circuit size.

Theorem 4 (SOP Network Correctness). *Let $G_{\text{SOP}}(f)$ be the series-parallel network with one N -transistor series chain per $\pi \in P^+$, all sharing SRC/SNK . Then $G_{\text{SOP}}(f)$ correctly implements f .*

Proof. *Coverage:* For $\pi \in P^+$, all literals in the chain evaluate to 1, forming a complete path. *Safety:* For $\pi' \in P^-$ and chain $\pi \in P^+$, since $\pi' \neq \pi$ they differ at some bit j^* , so the j^* -th transistor does not conduct under π' , breaking every path. $\square$

Theorem 5 (Completeness of Safety-Masked Search). *For any valid network G^* implementing f , every permutation of $E(G^*)$ is a valid trajectory under the safety mask. Equivalently, the safety-masked MDP reaches G^* from the empty graph in $|E(G^*)|$ steps following any edge ordering.*

Proof. Let $e_1, \dots, e_T$ be any ordering of $E(G^*)$ and $G_t = \{e_1, \dots, e_t\}$. We show by induction that G_t is safe for all t . *Base:* $G_0 = \emptyset$ is trivially safe. *Step:* Assume G_{t-1} is safe. Since $G_t \subseteq G^*$, any P^- -path in G_t would persist in G^* (by Theorem 2), contradicting G^* 's safety. So G_t is safe, and by Corollary 3, e_t is never rejected by the safety mask. $\square$

Implication for search completeness. Theorem 5 is the key correctness guarantee of our framework: *the safety mask never prunes any valid solution*. It only forbids transistors whose addition immediately creates an off-pattern path — those that cannot appear in any correct, safe network. The policy therefore searches the full space of valid partial sub-networks; no optimal solution is made unreachable by the masking procedure.

MDP Formulation

State. The state at step t is $\mathcal{G}_t = G_t \cup C$, where G_t is the partial synthesized network and $C = G_{\text{SOP}}(f)$ is the fixed compensate network from Theorem 4. The two share only SRC and SNK.

Action. Action $a_t = (u, v, \ell)$ adds one transistor. Two masks apply: (i) **reachability mask** on u prunes structurally disconnected nodes; (ii) **safety mask** on (u, v, ℓ) forbids transistors creating P^- -paths (Corollary 3). By Theorem 5, the safety mask is complete.

Reward. Terminal reward only (no per-step penalty):

$$r = \begin{cases} T^*(f)/T & \text{Phase 2: complete network, } T \text{ transistors} \\ \Phi^*(f)/\Phi & \text{Phase 3: complete PDN+PUN pair} \\ 0 & \text{incomplete.} \end{cases} \quad (1)$$

$T^*(f)$ and $\Phi^*(f)$ are self-tightening best values found so far for f .

Unified Graph State Representation

At each step t , the policy observes the unified graph $\mathcal{G}_t = G_t \cup C$, where:

- $G_t = (V_t, E_t)$ is the *partial synthesized network* built so far (edges are placed transistors with literal labels).
- $C = G_{\text{SOP}}(f)$ is the *fixed compensate network* (Theorem 4), kept constant across all steps of the episode.

The two components share only SRC and SNK; all internal nodes are disjoint. This unified representation gives the GNN simultaneous access to the current circuit topology (G_t) and the full logical specification of f (C), enabling the policy to reason jointly about what has been built and what coverage obligations remain.

Node and edge features. Each node $v \in \mathcal{G}_t$ carries a feature vector encoding: node type (SRC, SNK, G -internal, C -internal), connectivity degree in G_t and in C separately, and a coverage indicator (whether v lies on a chain for an already-covered on-pattern). Each edge $(u, v, \ell) \in E_t \cup E_C$ carries a feature vector encoding: component membership (G vs. C), variable index, and polarity. Node feature dimension d_{node} and edge feature dimension d_{edge} are given in Table 1.

Virtual new-node embedding. The node space is extended to $V_t^+ = V_t \cup \{v_{\text{new}}\}$ by appending a single learnable embedding $e_{\text{new}} \in \mathbb{R}^d$ for the virtual new-node option. This allows the policy to propose a fresh internal node at any step without a separate node-count decision, keeping the action space consistent regardless of how many nodes have been created so far.

Switching Graph Policy Network: Five-Head Factored Policy

The SGPN uses an MPNN backbone [Gilmer et al.(2017)Gilmer, Schütt, Mayr et al.] with L message-passing layers and hidden dimension d to compute node and graph-level features from $\mathcal{G}_t$. Let $\mathbf{h}_i \in \mathbb{R}^d$ be the node feature for node i and $\mathbf{g} \in \mathbb{R}^{2d}$ the graph-level summary. Four specialized MLP heads decode the transistor-placement action from these features, each operating over a different sub-decision of the factored action space. All four heads share the same MPNN forward pass; safety masks are applied independently within each head.

Head 1: Node-1 (source node). Scores each G -node $u \in V_t^+$ as the source of the new transistor:

$$s_u = \text{MLP}_1([\mathbf{h}_u \parallel \mathbf{g}]).$$

Logits are masked to allow only nodes in the reachability set.

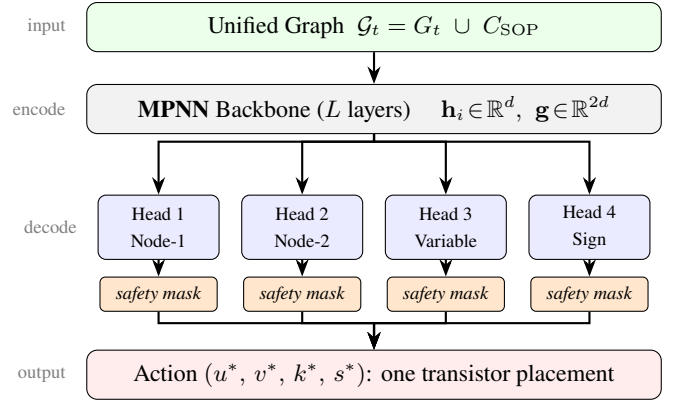


Figure 1: Architecture of the Switching Graph Policy Network (SGPN). The unified graph $\mathcal{G}_t = G_t \cup C_{\text{SOP}}$ is encoded by a shared MPNN backbone; four factored heads with per-head safety masks decode the transistor-placement action (u^*, v^*, k^*, s^*) .

Head 2: Node-2 (drain node). Conditioned on the sampled source u^* , scores each node $v \in V_t^+$, $v \neq u^*$:

$$s_v = \text{MLP}_2([\mathbf{h}_{u^*} \parallel \mathbf{h}_v \parallel \mathbf{g}]).$$

Logits are masked to exclude nodes that have no safe literal for any (u^*, v, ℓ) (precomputed BFS safety check).

Head 3: Variable index. Given the sampled edge (u^*, v^*) , selects the control variable:

$$\mathbf{s}_{\text{var}} = \text{MLP}_3([\mathbf{h}_{u^*} \parallel \mathbf{h}_{v^*}]) \in \mathbb{R}^{N_{\text{var}}}.$$

Logits are safety-masked: only variables with at least one safe polarity for (u^*, v^*) are valid.

Head 4: Polarity (sign). Selects positive or negative literal for the chosen variable:

$$\mathbf{s}_{\text{sgn}} = \text{MLP}_4([\mathbf{h}_{u^*} \parallel \mathbf{h}_{v^*}]) \in \mathbb{R}^2.$$

Logits are safety-masked per the chosen variable.

Factored log-probability. The log-probability of placing transistor (u, v, k, s) factorizes as:

$$\log \pi(a \mid \mathcal{G}_t) = \underbrace{\log p_1(u)}_{\text{Head 1}} + \underbrace{\log p_2(v \mid u)}_{\text{Head 2}} + \underbrace{\log p_3(k \mid u, v)}_{\text{Head 3}} + \underbrace{\log p_4(s \mid u, v, k)}_{\text{Head 4}} \quad (2)$$

where all probabilities are masked softmax over safe candidates. By Corollary 3, every action sampled from this distribution maintains the safety invariant; no post-hoc filtering is required.

Batched forward pass. During PPO training, K trajectories are collected per function per update step, each of length up to T_{max} . Naively, this requires $O(K \cdot T_{\text{max}})$ separate MPNN forward passes. Instead, all step graphs across all trajectories and all functions in the mini-batch are packed into a single `PackedGraph` and processed with *one* MPNN call. Per-step node features are then sliced from the shared output tensor for the head MLPs. This reduces the dominant computation cost by $K \cdot T_{\text{max}}$ and makes large-batch PPO training feasible on a single GPU.

Table 1: Architecture and training hyperparameters.

Hyperparameter	Value
MPNN hidden dim d	64
MPNN layers L	3
Head MLP hidden dim	128
Node feature dim d_{node}	[TODO:]
Edge feature dim d_{edge}	[TODO:]
Phase 1 lr	10^{-3}
Phase 1 batch size	32
Phase 1 epochs	[TODO:]
Phase 2 lr	3×10^{-5}
Phase 2 trajectories K	8
PPO clip ε	0.2
Entropy coeff λ_H	0.01
Agent sync interval M	5
Max steps T_{max}	30
Phase 3 lr	3×10^{-5}
Phase 3 pairs K	8
ASTRAN SA quality	1
ASTRAN place weights $(w_c, w_m, w_w, w_d, w_g)$	(3, 4, 1, 4, 2)
ASTRAN parallel workers	4

Phase 1: NLL Pretraining

We collect a dataset $\mathcal{D}$ of known-optimal transistor networks, each paired with its target Boolean function. For each network, we replay the transistor-placement sequence and record the (u, v, k, s) labels at every step. The pretraining objective is the mean negative log-likelihood:

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{|\mathcal{D}|} \sum_{(G^*, f) \in \mathcal{D}} \sum_{t=1}^{|E^*|} \log \pi(a_t^* | \mathcal{G}_{t-1}),$$

where a_t^* is the ground-truth transistor at step t . The unified graph $\mathcal{G}_{t-1}$ is constructed from the partial prefix G_{t-1}^* and the fixed compensate network C_f . Pretraining is batched via `PackedGraph` (all samples in a mini-batch share a single MPNN forward pass) and trained with Adam until convergence.

Phase 2: PPO Finetuning

After pretraining, we finetune with off-policy PPO. A frozen copy of the current model (*agent*) generates trajectories; the live model recomputes log-probabilities for the PPO surrogate loss.

Reward design. We use a purely terminal reward with no per-step penalty. Let T be the number of transistors placed and T^* the best transistor count found so far for this function (*adaptive self-tightening target*):

$$r = \begin{cases} T^*/T & \text{if complete,} \\ 0 & \text{otherwise.} \end{cases}$$

Once the policy discovers a shorter circuit, T^* updates automatically, so subsequent solutions that are longer receive a reduced reward. Each step in the trajectory receives the same scalar r (no discounting).

Algorithm 1 TransSGPN / SGPN Training (Phases 1–2)

Require: Dataset $\mathcal{D}$, function list $\mathcal{F}$, hyperparameters $K, M, \lambda_H, \varepsilon_{\text{clip}}$

- 1: Pretrain policy π_θ by NLL on $\mathcal{D}$ (Phase 1)
- 2: Initialize frozen agent $\pi_{\text{old}} \leftarrow \pi_\theta$
- 3: **for** each epoch **do**
- 4: Sample batch of functions $\mathcal{B} \subseteq \mathcal{F}$
- 5: **for** each $f \in \mathcal{B}$ **do**
- 6: Generate K trajectories using π_{old}
- 7: Update $T^*(f)$ if shorter complete circuit found
- 8: Compute rewards $\{r_k\}$ and GRPO advantages $\{\hat{A}_k\}$
- 9: Compute batched PPO + entropy loss
- 10: Update θ via Adam
- 11: **if** step $\equiv 0 \pmod{M}$ **then** $\pi_{\text{old}} \leftarrow \pi_\theta$

GRPO baseline. For each function in the batch, we sample K trajectories in parallel. The advantage for each trajectory is normalized within the group:

$$\hat{A}_k = \frac{r_k - \bar{r}}{\sigma_r + \varepsilon}, \quad \bar{r} = \frac{1}{K} \sum_{k=1}^K r_k, \quad \sigma_r = \sqrt{\frac{1}{K} \sum_{k=1}^K (r_k - \bar{r})^2}.$$

This group-relative policy optimization (GRPO) [Shao et al.(2024)Shao, Wang, Zhu et al.] baseline eliminates EMA lag and gives a strong positive signal to any trajectory shorter than its group average without requiring a value network.

Entropy bonus. To prevent mode collapse (where all K trajectories converge to the same solution), we add an entropy regularization term to the PPO loss:

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} - \lambda_H \cdot \bar{H},$$

where $\bar{H}$ is the mean per-step policy entropy across all four heads and λ_H is a tunable coefficient (default 0.01). The entropy bonus maintains exploration, ensuring the policy continues to occasionally sample shorter-than-current-best circuits.

Batched MPNN forward. To reduce wall-clock time, all trajectory steps across all K trajectories and all functions in the batch are packed into a single `PackedGraph` and processed with one MPNN forward pass. Subsequent head MLPs slice per-step node features from the shared output. This replaces $O(K \cdot T_{\text{max}})$ sequential forward passes with 1.

Agent synchronization. The frozen agent is re-synchronized to the live model every M PPO steps to keep the importance-sampling ratio π/π_{old} within a controlled range.

Phase 3: Physical-Aware Joint PDN+PUN Synthesis

Phase 2 optimizes transistor count for a single switching network. Phase 3 extends the RL loop to *joint* synthesis of the pull-down network (PDN) and pull-up network (PUN) of a complete CMOS cell, with a physical placement reward from ASTRAN [Ziesemer and Reis(2014)].

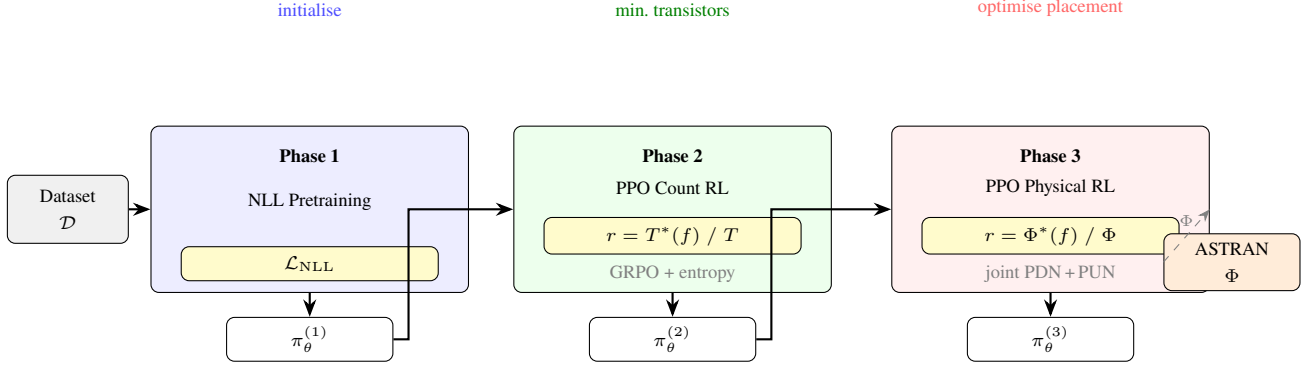


Figure 2: Three-phase training pipeline of TransSGPN. Phase 1 initialises the SGPN by NLL imitation on known-optimal circuits. Phase 2 finetunes toward minimum transistor count via PPO with adaptive reward $r = T^*(f)/T$ and GRPO baseline. Phase 3 jointly synthesises PDN and PUN and uses the ASTRAN placement objective Φ directly as the reward, optimising physical layout quality.

PDN and PUN function derivation. For a CMOS cell implementing $f(\mathbf{x})$, both networks must be synthesized simultaneously:

- **PDN** (NMOS, pulls output to GND when $f = 0$): switching function = $\neg f(\mathbf{x})$; on-patterns = P^- of f .
- **PUN** (PMOS, pulls output to VDD when $f = 1$): switching function = $f(\neg \mathbf{x})$, because PMOS transistors conduct when gate = 0; on-patterns = $\{\neg \pi : \pi \in P^+\}$.

Both are presented to the *same* TransSGPN policy as distinct switching-network synthesis problems: the policy is conditioned on on/off patterns and is agnostic to transistor type (NMOS/PMOS).

SPICE generation. A complete PDN+PUN pair is encoded as a standard SPICE `.subckt`: NMOS transistors (NMOS_VTL, $W = 90$ nm, $L = 50$ nm) implement the PDN with source at GND, drain at the output ZN, and internal nodes `ndk`; PMOS transistors implement the PUN symmetrically. Gate signals map directly from literal labels: positive literal $x_i \rightarrow$ port x_i ; negative literal $\neg x_i \rightarrow$ complemented port x_i_N . Power nets are named VCC/GND to match ASTRAN’s default supply identifiers.

Physical reward via ASTRAN. ASTRAN places the combined SPICE netlist using Threshold Accepting (parameters: fold 2 0, SA quality 1, 1 attempt). It returns five placement metrics on the “Final cost” line: cell width W_{cpp} (CPP = total poly columns), gate mismatches N_{gm} , wire length W_{wl} , routing density D_{rt} , and diffusion cuts N_{gap} . We define the physical objective as ASTRAN’s own weighted sum:

$$\Phi = w_c W_{\text{cpp}} + w_m N_{\text{gm}} + w_w W_{\text{wl}} + w_d D_{\text{rt}} + w_g N_{\text{gap}}, \quad (3)$$

with weights $(w_c, w_m, w_w, w_d, w_g) = (3, 4, 1, 4, 2)$ matching the ASTRAN SA cost function exactly, so minimizing Φ is equivalent to minimizing ASTRAN’s internal objective.

The joint reward for a complete PDN+PUN pair is:

$$r = \frac{\Phi^*(f)}{\Phi}, \quad \Phi^*(f) = \min_{\text{all pairs seen for } f} \Phi, \quad (4)$$

Algorithm 2 TransSGPN Phase 3: Physical-Aware Joint PDN+PUN RL

Require: Phase 2 policy π_{θ} , function list $\mathcal{F}$, ASTRAN bi-

```

nary, weights  $(w_c, w_m, w_w, w_d, w_g)$ 
1: Initialize  $\Phi^*(f) \leftarrow \infty$  for all  $f$ 
2:  $\pi_{\text{old}} \leftarrow \pi_{\theta}$ 
3: for each epoch do
4:   Sample batch  $\mathcal{B} \subseteq \mathcal{F}$ 
5:   for each  $f \in \mathcal{B}$  do
6:     for  $k = 1, \dots, K$  do
7:        $\tau_k^{\text{PDN}} \leftarrow \text{generate}(\pi_{\text{old}}, \neg f(\mathbf{x}))$ 
8:        $\tau_k^{\text{PUN}} \leftarrow \text{generate}(\pi_{\text{old}}, f(\neg \mathbf{x}))$ 
9:       if both complete then
10:         $\Phi_k \leftarrow \text{ASTRAN}(\text{SPICE}(\tau_k^{\text{PDN}}, \tau_k^{\text{PUN}}))$ 
11:        Update  $\Phi^*(f)$  if  $\Phi_k < \Phi^*(f)$ 
12:         $r_k \leftarrow \Phi^*(f) / \Phi_k$ 
13:       else  $r_k \leftarrow 0$ 
14:       GRPO:  $\hat{A}_k \leftarrow (r_k - \bar{r}) / (\sigma_r + \varepsilon)$ 
15:   Update  $\theta$  via PPO + entropy loss on all steps
16:   if step  $\equiv 0 \pmod{M}$  then  $\pi_{\text{old}} \leftarrow \pi_{\theta}$ 

```

where $\Phi^*(f)$ is the self-tightening best objective found so far. Incomplete pairs (either PDN or PUN fails to cover all on-patterns within T_{max} steps) receive $r = 0$.

Paired trajectory collection. For each function f per PPO step, K paired trajectories $(\tau_k^{\text{PDN}}, \tau_k^{\text{PUN}})_{k=1}^K$ are generated. ASTRAN is called only for complete pairs (up to K calls per function per step), and calls are parallelized across a thread pool. The GRPO advantage normalizes rewards within the K pairs: $\hat{A}_k = (r_k - \bar{r}) / (\sigma_r + \varepsilon)$. The same scalar $\hat{A}_k$ is applied to every step in both τ_k^{PDN} and τ_k^{PUN} : if the pair achieves better-than-average placement, both networks are reinforced; otherwise both are penalized.

Experiments

Setup

Dataset. [TODO: Describe how the dataset was generated: sweep over 3-variable Boolean functions, SAT-based minimum transistor count verification, etc.] Our dataset contains [TODO: N] Boolean functions of up to $N = 3$ input variables. For each function, the minimum transistor count is determined by a SAT-based solver [[TODO: Authors]([TODO: Year]b)], yielding ground-truth labels for pretraining and evaluation. The dataset is split into [TODO: X/Y/Z] train/validation/test functions.

Implementation. TransSGPN uses an MPNN backbone with $d = 64$, $L = 3$ layers, and MLP hidden dimension 128. Phase 1 runs for [TODO:] epochs ($\text{lr } 10^{-3}$, batch 32). Phase 2 uses $K = 8$ trajectories, $\text{lr } 3 \times 10^{-5}$, $\varepsilon = 0.2$, $\lambda_H = 0.01$, sync interval $M = 5$. Phase 3 initializes from the Phase 2 checkpoint, uses the same PPO hyperparameters, generates $K = 8$ paired PDN+PUN trajectories per function, and evaluates complete pairs with ASTRAN (freePDK45, SA quality 1, weights (3, 4, 1, 4, 2), 4 parallel workers). All experiments run on a single NVIDIA [TODO: GPU].

Baselines. We compare against:

- **MuSTNet** [[TODO: Authors]([TODO: Year]b)]: SAT-based exact synthesis.
- **MiniTNtk** [[TODO: Authors]([TODO: Year]a)]: heuristic transistor-network toolkit.
- **MCTS-PTNS** [[TODO: Authors](2026)]: Monte Carlo Tree Search synthesis.
- **TransSGPN Phase 1**: NLL pretraining only.
- **TransSGPN Phase 2**: transistor-count RL (single network).

For physical metrics (Phases 2–3), we generate the SPICE netlist for each synthesized PDN+PUN pair and evaluate it with ASTRAN using the same placement parameters to ensure a fair comparison.

Metrics. *Logical metrics* (Phases 1–2):

- **Success rate**: fraction of functions with a correct implementation.
- **Optimality rate**: fraction matching the known minimum transistor count.
- **Avg. transistors**: mean transistor count over correct solutions.

Physical metrics (Phase 3):

- **ASTRAN objective Φ** (Eq. 3): weighted sum of CPP, gate mismatches, WL, routing density, and diffusion cuts.
- **CPP** (cell width): total poly columns used by the placed cell.
- **Gate mismatches**: positions where P-row and N-row gate signals differ.
- **Nr. Gaps**: number of diffusion cuts (Euler-path breaks).
- **WL / Rt. Density**: wire length and peak routing congestion.

Main Results

[TODO: Table 1: comparison with baselines across all test functions.]

Table 2: Logical synthesis comparison ([TODO: N]-function test set).

Method	Success (%)	Optimality (%)	Avg. T	Time (s)
MuSTNet [[TODO: Authors]([TODO: Year]b)]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
MiniTNtk [[TODO: Authors]([TODO: Year]a)]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
MCTS-PTNS [[TODO: Authors](2026)]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
TransSGPN Phase 1	[TODO:]	[TODO:]	[TODO:]	[TODO:]
TransSGPN Phase 2 (ours)	[TODO:]	[TODO:]	[TODO:]	[TODO:]

Table 3: Ablation on the XOR-3 function (20 trials per variant).

Variant	Success (%)	Avg. T
EMA baseline	[TODO:]	[TODO:]
GRPO baseline (ours)	[TODO:]	[TODO:]
Fixed t_{ref}	[TODO:]	[TODO:]
Adaptive t^* reward (ours)	[TODO:]	[TODO:]
No entropy ($\lambda_H = 0$)	[TODO:]	[TODO:]
Entropy $\lambda_H = 0.01$ (ours)	[TODO:]	[TODO:]
Full TransSGPN	[TODO:]	[TODO:]

Table 2 reports the main comparison. TransSGPN achieves a success rate of [TODO: X%] and matches the minimum transistor count on [TODO: Y%] of successfully synthesized functions, outperforming the heuristic baseline MiniTNtk and approaching the exact SAT-based MuSTNet. Unlike MuSTNet, TransSGPN requires no per-instance SAT invocation at inference and runs in [TODO: X ms] per function on average.

Ablation Study

Table 3 ablates the key design choices in Phase 2 training.

Effect of GRPO baseline. [TODO: Discuss: EMA baseline causes slow convergence because the baseline lags behind the policy. GRPO instantly adapts to the current batch.]

Effect of adaptive reward. [TODO: Discuss: once the policy finds a shorter circuit, the self-tightening target T^* reduces reward for longer solutions, maintaining gradient signal toward the minimum even in batches where no new best is found.]

Effect of entropy bonus. [TODO: Discuss: without entropy, the policy mode-collapses to a single 9T solution and ppo_loss goes to zero. With $\lambda_H = 0.01$ the policy maintains diversity and eventually discovers 8T solutions.]

Phase 3: Physical-Aware Placement Results

Setup. For each method, we generate the SPICE netlist for the best PDN+PUN pair found over 20 trials and evaluate it with ASTRAN (freePDK45, same placement parameters as training). The physical RL baseline starts from the Phase 2 checkpoint and is finetuned for [TODO:] epochs.

Physical RL reduces placement cost. [TODO: Discuss: Phase 3 reduces Φ by X% vs Phase 2 on XOR-3. Key gains: CPP drops from Y to Z (fewer poly columns), gate

Table 4: Physical placement comparison on XOR-3 (20 generation trials). Φ is the ASTRAN weighted objective (Eq. 3); lower is better.

Method	Φ	CPP	GM	WL	Gaps
TransSGPN Phase 2 (no physical RL)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
TransSGPN Phase 3 (ours)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

mismatches reduced to near-zero. Nr. Gaps improvement quantifies better dual Euler path quality. Concrete result from training log: best_obj improved from 159 (Phase 2 baseline on NET.sp) to [TODO:].

CPP and Width relationship. For a cell with T_{PDN} and T_{PUN} transistors, cell width satisfies:

$$CPP = 2 \cdot \max(T_{PDN}, T_{PUN}) + 1 + (\# \text{ unique gap positions}).$$

For our XOR-3 result (8+8 transistors), the minimum achievable CPP is 17 (perfect dual Euler path, zero gaps); Phase 3 achieves CPP=18 with Nr. Gaps=2, just one column above the theoretical minimum.

Generalization to Unseen Functions

[TODO: Show that a policy trained on a subset of functions generalizes to held-out functions without retraining.]

Scaling Analysis

[TODO: Show how inference time scales with number of variables. Compare against SAT-based methods.]

Conclusion

We presented TransSGPN, a three-phase learning-based framework for transistor network synthesis. The key technical contributions are: (i) a unified graph representation combining the partial synthesized network with a fixed SOP compensate network, together with formal safety and correctness guarantees that make every generated trajectory valid by construction; (ii) a five-head factored policy with safety-masked actions operating on this unified graph; (iii) a two-phase logical training procedure (NLL pretraining + PPO with GRPO baseline, adaptive self-tightening reward, and entropy regularization) that converges toward minimum transistor count; and (iv) a physical-aware Phase 3 that jointly synthesizes PDN and PUN, evaluates each complete pair with ASTRAN, and uses the weighted placement objective $\Phi = w_c \cdot CPP + w_m \cdot GM + w_w \cdot WL + w_d \cdot D + w_g \cdot \text{Gaps}$ directly as the RL reward, achieving physical quality improvement without separate post-synthesis optimization.

Experiments demonstrate that a single learned policy generalizes across diverse Boolean functions, approaches exact SAT-based transistor count, and — through Phase 3 physical RL — produces PDN+PUN pairs with competitive ASTRAN placement objectives at a fraction of the per-instance cost of search-based methods.

Future work. Extending TransSGPN to four- and five-variable Boolean functions, incorporating transistor sizing into the policy, and closing the loop with full routing metrics (via ASTRAN compact and ILP reranking) are natural next steps.

References

- [Gilmer et al.(2017)] Gilmer, J.; Schütt, K. T.; Mayr, A.; et al. 2017. Neural Message Passing for Quantum Chemistry. *International Conference on Machine Learning (ICML)*.
- [Shao et al.(2024)] Shao, Z.; Wang, P.; Zhu, Q.; et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.
- [[TODO: Authors](2026)] [TODO: Authors]. 2026. MCTS-PTNS: General and Scalable Physical-Aware Transistor Network Synthesis via Monte Carlo Tree Search. *International Conference on Computer-Aided Design (ICCAD)*.
- [[TODO: Authors]([TODO: Year]a)] [TODO: Authors]. [TODO: Year]a. MiniTNtk: [TODO: Full title]. [TODO: Venue].
- [[TODO: Authors]([TODO: Year]b)] [TODO: Authors]. [TODO: Year]b. MuSTNet: SAT-Based Exact Multi-Stage Transistor Network Synthesis with Placement Awareness. [TODO: Venue].
- [Ziesemer and Reis(2014)] Ziesemer, A.; and Reis, R. 2014. ASTRAN: Automatic Synthesis of Transistor Networks. In *IEEE International Symposium on Integrated Circuits and Systems (ISICAS)*. Open-source standard-cell layout framework; <https://github.com/aziesemer/astran>.